

04-异常处理器代码片段

```
1  import traceback
2
3  from fastapi import HTTPException, Request
4  from fastapi.responses import JSONResponse
5  from sqlalchemy.exc import IntegrityError, SQLAlchemyError
6  from starlette import status
7
8  # 开发模式: 返回详细错误信息
9  # 生产模式: 返回简化错误信息
10 DEBUG_MODE = True # 教学项目保持开启
11
12
13 ▼ async def http_exception_handler(request: Request, exc: HTTPException):
14     """
15     处理 HTTPException 异常
16     """
17     # HTTPException 通常是业务逻辑主动抛出的, data 保持 None
18     return JSONResponse(
19         status_code=exc.status_code,
20         content={
21             "code": exc.status_code,
22             "message": exc.detail,
23             "data": None
24         }
25     )
26
27
28 ▼ async def integrity_error_handler(request: Request, exc: IntegrityError):
29     """
30     处理数据库完整性约束错误
31     """
32     error_msg = str(exc.orig)
33
34     # 判断具体的约束错误类型
35 ▼     if "username_UNIQUE" in error_msg or "Duplicate entry" in error_msg:
36         detail = "用户名已存在"
37 ▼     elif "FOREIGN KEY" in error_msg:
38         detail = "关联数据不存在"
39 ▼     else:
40         detail = "数据约束冲突, 请检查输入"
41
42     # 开发模式下返回详细错误信息
43     error_data = None
44 ▼     if DEBUG_MODE:
45 ▼         error_data = {
```

```

46         "error_type": "IntegrityError",
47         "error_detail": error_msg,
48         "path": str(request.url)
49     }
50
51     return JsonResponse(
52         status_code=status.HTTP_400_BAD_REQUEST,
53         content={
54             "code": 400,
55             "message": detail,
56             "data": error_data
57         }
58     )
59
60
61 async def sqlalchemy_error_handler(request: Request, exc: SQLAlchemyError
62 ):
63     """
64     处理 SQLAlchemy 数据库错误
65     """
66     # 开发模式下返回详细错误信息
67     error_data = None
68     if DEBUG_MODE:
69         error_data = {
70             "error_type": type(exc).__name__,
71             "error_detail": str(exc),
72             "traceback": traceback.format_exc(),
73             "path": str(request.url)
74         }
75
76     return JsonResponse(
77         status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
78         content={
79             "code": 500,
80             "message": "数据库操作失败, 请稍后重试",
81             "data": error_data
82         }
83     )
84
85 async def general_exception_handler(request: Request, exc: Exception):
86     """
87     处理所有未捕获的异常
88     """
89     # 开发模式下返回详细错误信息
90     error_data = None
91     if DEBUG_MODE:
92         error_data = {

```

```
93         "error_type": type(exc).__name__,
94         "error_detail": str(exc),
95         # 格式化异常信息为字符串，方便日志记录和调试
96         "traceback": traceback.format_exc(),
97         "path": str(request.url)
98     }
99
100     return JsonResponse(
101         status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
102         content={
103             "code": 500,
104             "message": "服务器内部错误",
105             "data": error_data
106         }
107     )
108
109
110
111
```